```
end
```

This works as long as our model classes don't contain any additional functionality that would have been used in the migration—all we're creating here is a bare-bones version.

Defining Indices

Migrations can (and probably should) define indices for tables. For example, we might notice that once our application has a large number of orders in the database, searching based on the customer's name takes longer than we'd like. It's time to add an index using the appropriately named `add_index` method:

```
class AddCustomerNameIndexToOrders < ActiveRecord::Migration
  def change
    add_index :orders, :name
  end
end
```

If we give `add_index` the optional parameter `unique: true`, a unique index will be created, forcing values in the indexed column to be unique.

By default the index will be given the name *index_table_on_column*. We can override this using the `name: "somename"` option. If we use the `:name` option when adding an index, we'll also need to specify it when removing the index.

We can create a *composite index*—an index on multiple columns—by passing an array of column names to `add_index`.

Indices are removed using the `remove_index` method.

Primary Keys

Rails assumes every table has a numeric primary key (normally called `id`) and ensures the value of this column is unique for each new row added to a